

Bayesian Reinforcement Learning for Warehouse Path Planning: A Comparison with Classical Planners

Parham Khoshsolat · Advanced Statistics, Mode A

May 23, 2026

Abstract. We replicate the deep-exploration idea of Osband et al. (2016, *Deep Exploration via Bootstrapped DQN*) and apply it to a simulated warehouse environment inspired by the Challenge Lab “RL for Logistic Optimization” specification. Concretely, we model a single Laser Guided Vehicle (LGV) operating in a randomly-laid-out warehouse with discrete kinematic constraints (turn-left / turn-right / forward), and compare four algorithms: A*, Dijkstra, vanilla DQN, and Bootstrapped DQN ($K = 10$ ensemble heads acting as an approximate posterior over Q-functions). The Bayesian flavor is provided by the bootstrap ensemble — each head sees a different subset of the replay buffer, and one head is committed to per episode, giving *coherent* exploration. We report training curves, eval success rates, and a statistical comparison.

1. Introduction

1.1. Context

Warehouses with autonomous vehicles (Laser-Guided Vehicles, LGVs) are an active deployment area for AI in logistics. The Challenge Lab specification for this course project (provided by the instructor) calls for an RL-based path-planning system: an agent must learn to pick up pallets, deliver them to drop zones, and avoid collisions. Performance must be compared against classical planning algorithms such as A* and Dijkstra.

A core difficulty in this setting is *exploration*: the agent has to discover which actions lead to reward (pickup, delivery) in an environment where most actions yield only a small step cost. The classical solution — ϵ -greedy random exploration — is known to be inefficient in sparse-reward settings.

1.2. Contribution

This project’s contribution is methodological rather than algorithmic. We:

1. **Implement a parameterised warehouse simulator** (Gymnasium API, single- and multi-agent, configurable layout / shelves / pickups / drops).
2. **Compare four planning/learning approaches** head-to-head on the same env: A*, Dijkstra, vanilla DQN, and Bootstrapped DQN.
3. **Analyse the failure modes** of vanilla deep RL on the prof’s full specification (12×12 grid, 3 pickups), and **scale down** to a setting (8×8 grid, 1 pickup) where the comparison is informative.
4. **Quantify the Bayesian posterior diversity** via the ensemble’s per-state Q-value standard deviation across training.

We do *not* claim a novel algorithm. The contribution is a clean, fully reproducible empirical comparison with heavy diagnostic instrumentation.

1.3. Paper outline

Section 2 reviews related work. Section 3 describes the warehouse env, the classical baselines, and the two deep-RL algorithms. Section 4 details our experimental design. Section 5 presents the results. Section 6 discusses limitations and future work.

2. Related work

Deep Q-Networks (DQN). Mnih et al. (2015) demonstrated that a deep convolutional Q-network with experience replay and target network can play Atari games at human level [Mnih et al., 2015]. We use the same algorithm template (replay buffer + target network + Bellman backup) on our gridworld observations.

Bootstrapped DQN. Osband et al. (2016) replaced ε -greedy exploration with posterior sampling over a bootstrap ensemble of Q-heads [Osband et al., 2016]. The intuition: each head is trained on a different bootstrap sample of experience and approximates a draw from a posterior over the value function. Committing to one head per episode provides *coherent* deep exploration. This is the anchor paper of our project.

Multi-Agent Path Finding via RL. Sartoretti et al. (2019, PRIMAL) trained multi-agent path-finding agents via a combination of RL and imitation of an expert planner [Sartoretti et al., 2019]. PRIMAL is the conceptual SOTA for our warehouse domain; we do not replicate it (too compute-intensive for our 5-day scope), but cite it as future work.

3. Materials and methods

3.1. Warehouse environment

(docs/example_layout.png will be inserted by build script)

Listing 1: Example 12×12 layout. # are shelves, P = pickup zones, D = drop zones, agent (blue triangle) faces N/E/S/W.

The environment is a square gridworld of side $L \in \{8, 12\}$. Each cell is either empty, a shelf (impassable), a pickup zone (carries a pallet), or a drop zone. One or more LGVs occupy distinct cells.

State. The full state of the environment is the grid layout plus, per agent, position (r, c) , facing $f \in \{N, E, S, W\}$, and a binary *carrying* flag.

Observation. Each agent sees a $(2w + 1) \times (2w + 1)$ egocentric window of cell types (one-hot over 5 channels: empty / shelf / pickup / drop / other-agent), plus its own facing (one-hot, 4 dims) plus its carrying flag (1 dim). For $w = 5$ (our default) this is 610 features per agent.

Actions. Three discrete actions per agent: turn left, turn right, move forward. Two of three actions cause no displacement — this discretely models the prof’s “kinematic constraint” requirement.

Reward (per step, per agent).

$$r_t = \underbrace{-1}_{\text{step}} + \underbrace{+10}_{\text{pickup}} + \underbrace{+25}_{\text{delivery}} + \underbrace{-5}_{\text{collision}} + \underbrace{F(s_t, s_{t+1})}_{\text{shaping}}$$

The shaping term F is the potential-based shaping of Ng, Harada, Russell (1999), with potential $\Phi(s) = -d(s, \text{nearest relevant target})$, clipped to ± 0.5 per step to bound the rare carry-flag flip contribution. Potential-based shaping is theoretically guaranteed to preserve the optimal policy.

Termination. Episode terminates when all pickups are picked *and* delivered. Truncates after `max_steps` steps.

3.2. Classical baselines

We implement A* and Dijkstra over the joint state space (position $\times$ facing), with the same three actions used by the RL agent — so the resulting path costs are directly comparable. A* uses the Manhattan distance heuristic (admissible on uniform grids). Multi-agent variants use Cooperative A* with a time-indexed reservation table.

3.3. Vanilla DQN

We use a standard DQN with: MLP Q-network (2 hidden layers of 128 units, ReLU), uniform replay buffer (100k transitions), target network with hard update every 500 gradient steps, Adam optimiser at 5×10^{-4} , Smooth L1 loss, gradient clipping at $\|g\|_2 = 10$, $\gamma = 0.99$, batch size 64, ε -greedy decaying linearly from 1.0 to 0.05 over the first half of training.

3.4. Bootstrapped DQN

The Bayesian twist. The Q-network shares an MLP torso and exposes $K = 10$ parallel linear heads. The replay buffer stores a K -dimensional bootstrap mask $m \in \{0, 1\}^K$ per transition (each entry an independent Bernoulli(0.5)). The per-head loss is masked accordingly:

$$\mathcal{L} = \left(\frac{1}{\sum_{i,k} m_{i,k}} \right) \sum_{i,k} m_{i,k} \cdot \text{Huber} \left(Q_k(s_i, a_i) - \left(r_i + \gamma \max_{a'} Q_k^{\text{target}}(s'_i, a') \right) \right)$$

At the *start* of every episode we sample one head uniformly and commit to it for action selection until the episode ends. This is what gives Bootstrapped DQN its “coherent” exploration property — within an episode, the agent acts as if it confidently believes one approximate-posterior sample.

For evaluation we take the argmax of the *mean* of the K heads’ Q-values — a committee vote.

4. Experimental design

4.1. Environments

We run experiments on two configurations:

Config	Grid	Pickups / drops	Note
<i>easy</i>	8×8	1 / 1	main comparison setting
<i>hard</i> (prof spec)	12×12	3 / 2	scaling test, “future work”

The *easy* env is the main comparison: vanilla DQN learns it, leaving room for the Bayesian twist to improve. The *hard* env tests whether either deep-RL method can scale to the full prof spec.

4.2. Algorithms compared

Algorithm	Description
A*	classical planner with Manhattan heuristic
Dijkstra	classical planner, $h \equiv 0$
DQN	vanilla deep RL with ε -greedy exploration
Bootstrapped DQN ($K = 10$)	Bayesian posterior-sampling exploration

4.3. Metrics

For each algorithm we report, on 50 held-out episodes:

- **success rate**: fraction of episodes where all pickups are delivered before truncation,
- **mean episode reward** and standard deviation,
- **mean episode length** (steps),
- **mean collision count**.

For statistical comparison between vanilla and Bootstrapped DQN we use Welch's t -test on the final-eval rewards across 5 random seeds.

5. Results

(To be filled in once Day 3 / Day 4 experiments complete.)

6. Discussion

(To be filled in.)

7. Conclusion and future work

(To be filled in.)

8. Contributions

Parham Khoshsolat — sole author of this Mode A submission. Designed the environment, implemented all four algorithms, ran all experiments, built the interactive demo web app at <https://github.com/parhamkhoshsolat/warehouse-bayesian-rl>, wrote this report.

9. References

- Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). *Human-level control through deep reinforcement learning*. Nature.
- Osband, I., Blundell, C., Pritzel, A., & Van Roy, B. (2016). *Deep Exploration via Bootstrapped DQN*. NeurIPS.
- Sartoretti, G., et al. (2019). *PRIMAL: Pathfinding via Reinforcement and Imitation Multi-Agent Learning*. IEEE RA-L.
- Ng, A. Y., Harada, D., & Russell, S. (1999). *Policy invariance under reward transformations: Theory and application to reward shaping*. ICML.
- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). *A formal basis for the heuristic determination of minimum cost paths*. IEEE Transactions on Systems Science and Cybernetics.
- Dijkstra, E. W. (1959). *A note on two problems in connexion with graphs*. Numerische Mathematik.